Lecture 16 – First-Class Continuations

COSE212: Programming Languages

Jihyeok Park



2026 Fall

Recall

- We will learn about **continuations** with the following topics:
  - **Continuations** (Lecture 14 & 15)
  - **First-Class Continuations** (Lecture 16)
  - **Compiling with continuations** (Lecture 17)

- A **continuation** represents the **rest of the computation**.
  - Continuation Passing Style (CPS)
  - Interpreter of FAE in CPS
  - Small-step operational semantics of FAE

- In this lecture, we will learn **first-class continuations** and how to define the **control flow changes** in a program using them.

- KFAE – FAE with **first-class continuations**
  - Interpreter and Small-Step semantics

Contents

1. First-Class Continuations

2. KFAE – FAE with First-Class Continuations
   - Concrete/Abstract Syntax

3. Interpreter and Small-Step Semantics for KFAE
   - Recall: Interpreter and Small-Step Semantics for FAE
   - Interpreter and Small-Step Semantics for KFAE
   - First-Class Continuations
   - Function Application
   - Example 1
   - Example 2

4. Control Statements

Contents

1. First-Class Continuations

2. KFAE – FAE with First-Class Continuations
    - Concrete/Abstract Syntax

3. Interpreter and Small-Step Semantics for KFAE
    - Recall: Interpreter and Small-Step Semantics for FAE
    - Interpreter and Small-Step Semantics for KFAE
    - First-Class Continuations
    - Function Application
    - Example 1
    - Example 2

4. Control Statements

Recall: First-Class Citizen

PLRG

In a programming language, an entity is said to be **first-class citizen** if it is treated as a **value**. In other words, it can be

1. **assigned** to a **variable**,
2. **passed** as an **argument** to a function, and
3. **returned** from a function.

For example, Scala supports **first-class functions**.

```
def inc(n: Int): Int = n + 1
// 1. We can assign a function to a variable.
val f: Int => Int = inc
// 2. We can pass a function as an argument to a function.
List(1, 2, 3).map(inc)                    // List(2, 3, 4)
// 3. We can return a function from a function.
def addN(n: Int): Int => Int = m => n + m
val add3: Int => Int = addN(3)
add3(5)                                   // 3 + 5 = 8
```

First-class Continuations

Similarly, a **first-class continuation** means that a continuation is treated as a **value**. For example, `Racket` supports **first-class continuations**.

In `Racket`, we can 1) **capture** the continuation using `let/cc` and 2) **change** the program's **control flow** using the captured continuation.

For example, let's change the control flow of the following program:

```
; Racket
(* 2 (+ 3 5))
```

(Note that `Racket` uses the prefix notation (e.g., `(+ 1 2)`) instead of the infix notation (e.g., `1 + 2`).)

by using the `let/cc` (`let`-binding with **current continuation**) as follows:

```
; Racket
(* 2 (let/cc k (+ 3 (k 5))))   ; first-class continuation with `let/cc`
```

Let's compare the evaluation of the two expressions.

First-class Continuations

PLRG

The original expression is evaluated in the following order:

```
; Racket
(* 2 (+ 3 5))
```

1. Evaluate 2. (Result: 2)
2. Evaluate 3. (Result: 3)
3. Evaluate 5. (Result: 5)
4. Add the results of step 2 and 3. (Result: `3 + 5 = 8`)
5. Multiply the results of step 1 and 2 – 4. (Result: `2 * 8 = 16`)

What is the continuation of the expression `(+ 3 5)`? Step 5.

What is the continuation of the expression `5`? Steps 4 and 5.

Let's 1) **capture** the continuation of `(+ 3 5)` (i.e., 5) using `let/cc` and 2) **change** the **control flow** after evaluating `5` by using it as the continuation of `5` instead of the original one (i.e., 4 and 5).

First-class Continuations

PLRG

We can change the program's control flow as follows:

```
; Racket
(* 2 (let/cc k (+ 3 (k 5))))   ; first-class continuation with `let/cc`
```

❶	Evaluate 2.	(Result: 2)
❷	Let `k` be the continuation of ❷ – ❼.	(`k` is Step ❽)
❸	Evaluate 3.	(Result: 3)
❹	Evaluate `k`.	(Result: Step ❽)
❺	Evaluate 5.	(Result: 5)
❻	Call the result of step ❹ with that of ❺.	(**Replace Cont.**)
❼	Add the results of step ❸ and ❹ – ❻.	(Unreachable)
❽	Multiply the results of step ❶ and ❷ – ❼.	(Result: `2 * 5 = 10`)

It means that

- Step ❷ defines the continuation of steps ❷ – ❼ as a value in `k`.
- Step ❻ replaces the continuation of step ❺ with `k`.

First-class Continuations

Some functional languages support **first-class continuations**.

- Racket

```
(* 2 (let/cc k (+ 3 (k 5))))                ; 2 * 5 = 10
```

- Ruby

```
2 * (callcc { |k| 3 + k.call(5) })          # 2 * 5 = 10
```

- Haskell

```
do
  x <- callCC $ \k -> do
    y <- k 5
    return $ 3 + y
  return $ 2 * x                            -- 2 * 5 = 10
```

- ...

Contents

1. First-Class Continuations

2. KFAE – FAE with First-Class Continuations
    - Concrete/Abstract Syntax

3. Interpreter and Small-Step Semantics for KFAE
    - Recall: Interpreter and Small-Step Semantics for FAE
    - Interpreter and Small-Step Semantics for KFAE
    - First-Class Continuations
    - Function Application
    - Example 1
    - Example 2

4. Control Statements

KFAE – FAE with First-Class Continuations

Now, let's extend FAE into KFAE with a new keyword `vcc` to capture the **first-class continuations**.

```
/* KFAE */
2 * { vcc k; 3 + k(5) }
```

Here is another example of KFAE:

```
/* KFAE */
{
  vcc done;
  val f = {
    vcc exit;
    2 * done(1 + {
      vcc k;
      exit(k)
    })
  };
  f(3) * 5
}
```

Concrete/Abstract Syntax

PLRG

For KFAE, we need to extend **expressions** of FAE with

1. **first-class continuations** (`vcc`)

We can extend the **concrete syntax** of FAE as follows:

```
// expressions
<expr> ::= ... | "vcc" <id> ";" <expr>
```

and the **abstract syntax** of FAE as follows:

$$\text{Expressions} \quad \mathbb{E} \ni e ::= \ldots \mid \texttt{vcc}\ x;\ e \qquad (\texttt{Vcc})$$

```
enum Expr:
  ...
  // first-class continuations
  case Vcc(name: String, body: Expr)
```

KFAE – FAE with First-Class Continuations

Then, what is the expected result of the following KFAE expression?

```
/* KFAE */

2 * { vcc k; 3 + k(5) }
```

KFAE – FAE with First-Class Continuations

Then, what is the expected result of the following KFAE expression?

```
/* KFAE */
// k is a continuation that can be represented as `x => 2 * x`
2 * { vcc k; 3 + k(5) }
```

KFAE – FAE with First-Class Continuations

Then, what is the expected result of the following KFAE expression?

```
/* KFAE */
// k is a continuation that can be represented as `x => 2 * x`
2 * { vcc k; 3 + k(5) } // 2 * 5 = 10
```

`{ ... }` becomes 5

Compare it with a **function** that does the same thing:

```
/* FAE */

2 * { val k = x => 2 * x; 3 + k(5) } // 2 * (3 + 10) = 26
```

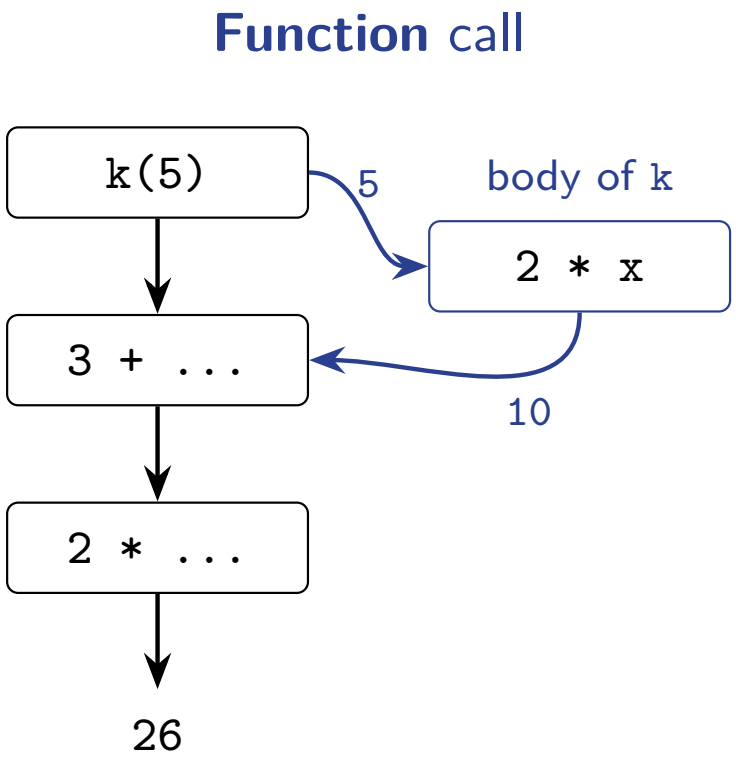
call

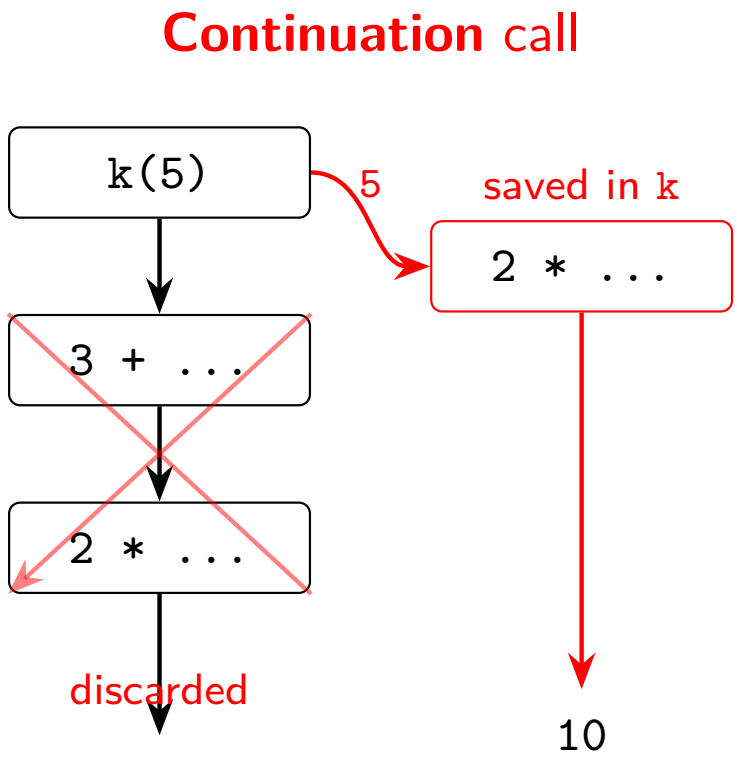
`k(5)` becomes 10

- A function call returns its result to where it was called.
- A continuation call never returns. Instead, the value of `{ vcc k; ... }` is replaced with its argument.

Function Call vs. Continuation Call

What happens to the **remaining computations** when calling `k(5)`?





KFAE – FAE with First-Class Continuations

Then, what is the expected result of the following KFAE expression?

```
/* KFAE */
{
  vcc done;
  val f = {
    vcc exit;
    2 * done(1 + {
      vcc k;
      exit(k)
    })
  };
  f(3) * 5
}
```

KFAE – FAE with First-Class Continuations

Then, what is the expected result of the following KFAE expression?

```
/* KFAE */
{
  vcc done;          // done = x => x
  val f = {
    vcc exit;
    2 * done(1 + {
      vcc k;
      exit(k)
    })
  };
  f(3) * 5
}
```

KFAE – FAE with First-Class Continuations

Then, what is the expected result of the following KFAE expression?

```
/* KFAE */
{
  vcc done;          // done = x => x
  val f = {
    vcc exit;        // exit = y => val f = y; f(3) * 5
    2 * done(1 + {
      vcc k;
      exit(k)
    })
  };
  f(3) * 5
}
```

KFAE – FAE with First-Class Continuations

Then, what is the expected result of the following KFAE expression?

```
/* KFAE */
{
  vcc done;          // done = x => x
  val f = {
    vcc exit;        // exit = y => val f = y; f(3) * 5
    2 * done(1 + {
      vcc k;         // k    = z => val f = { 2 * done(1 + z) }; f(3) * 5
      exit(k)
    })
  };
  f(3) * 5
}
```

KFAE – FAE with First-Class Continuations

Then, what is the expected result of the following KFAE expression?

```
/* KFAE */
{
  vcc done;          // done = x => x
  val f = {
    vcc exit;        // exit = y => val f = y; f(3) * 5
    2 * done(1 + {
      vcc k;         // k    = z => val f = { 2 * done(1 + z) }; f(3) * 5
      exit(k)
    })
  };
  f(3) * 5
}                    // 4 (= 1 + 3)
```

Contents

1. First-Class Continuations

2. KFAE – FAE with First-Class Continuations
   - Concrete/Abstract Syntax

3. Interpreter and Small-Step Semantics for KFAE
   - Recall: Interpreter and Small-Step Semantics for FAE
   - Interpreter and Small-Step Semantics for KFAE
   - First-Class Continuations
   - Function Application
   - Example 1
   - Example 2

4. Control Statements

Recall: Interpreter and Small-Step Sem. for FAE

In the previous lecture, we have defined the **first-order representation** of **continuations** with **value stack**:

```
enum Cont:
  case EmptyK
  case EvalK(env: Env, expr: Expr, k: Cont)
  case AddK(k: Cont)
  case MulK(k: Cont)
  case AppK(k: Cont)

type Stack = List[Value]
```

$$
\begin{array}{lrll}
\text{Continuations} & \mathbb{K} \ni \kappa ::= & \square & (\texttt{EmptyK}) \\
& & \mid (\sigma \vdash e) :: \kappa & (\texttt{EvalK}) \\
& & \mid (\texttt{+}) :: \kappa & (\texttt{AddK}) \\
& & \mid (\texttt{*}) :: \kappa & (\texttt{MulK}) \\
& & \mid (\texttt{@}) :: \kappa & (\texttt{AppK}) \\
\text{Value Stacks} & \mathbb{S} \ni s ::= & \blacksquare \mid v :: s & (\texttt{List[Value]})
\end{array}
$$

Recall: Interpreter and Small-Step Sem. for FAE

Then, we have defined the **small-step relation** $\rightarrow \subseteq (\mathbb{K} \times \mathbb{S}) \times (\mathbb{K} \times \mathbb{S})$ between **states** consisting of pairs of **continuations** and **value stacks**:

```
def step(k: Cont, s: Stack): (Cont, Stack) = ???
```

$$\langle \kappa \mid\mid s \rangle \rightarrow \langle \kappa' \mid\mid s' \rangle$$

And the `eval` function **iteratively steps** the state until it reaches the empty continuation $\square$ and returns the single value in the value stack:

```
def eval(str: String): String =
  import Cont.*
  def aux(k: Cont, s: Stack): Value = step(k, s) match
    case (EmptyK, List(v)) => v
    case (k, s) => aux(k, s)
  aux(EvalK(Map.empty, Expr(str), EmptyK), List.empty).str
```

$$\langle (\varnothing \vdash e) :: \square \mid\mid \blacksquare \rangle \rightarrow^* \langle \square \mid\mid v :: \blacksquare \rangle$$

Interpreter and Small-Step Semantics for KFAE

Now, let's extend the interpreter and small-step semantics for FAE to KFAE by adding the **first-class continuations**.

First, we need to extend the values of FAE with **continuation values** consisting of pairs of continuations and value stacks:

```
// values
enum Value:
  case NumV(number: BigInt)
  case CloV(param: String, body: Expr, env: Env)
  case ContV(cont: Cont, stack: Stack)
```

$$\begin{array}{lrll} \text{Values} & \mathbb{V} \ni v ::= & n & (\texttt{NumV}) \\ & & \mid \langle \lambda x.e, \sigma \rangle & (\texttt{CloV}) \\ & & \mid \langle \kappa \mid\mid s \rangle & (\texttt{ContV}) \end{array}$$

Then, let's fill out the missing cases in the `step` function and rules for $\rightarrow$ in the small-step semantics of KFAE.

First-Class Continuations

```
def step(k: Cont, s: Stack): (Cont, Stack) = (k, s) match
  case (EvalK(env, expr, k), s) => expr match
    ...
    case Vcc(x, b) => (EvalK(env + (x -> ContV(k, s)), b, k), s)
```

$$\langle \kappa \mid\mid s \rangle \rightarrow \langle \kappa \mid\mid s \rangle$$

$$\texttt{Vcc} \quad \langle (\sigma \vdash \texttt{vcc}\ x;\ e) :: \kappa \mid\mid s \rangle \quad \rightarrow \quad \langle (\sigma[x \mapsto \langle \kappa \mid\mid s \rangle] \vdash e) :: \kappa \mid\mid s \rangle$$

It defines a new immutable binding x in the environment σ that maps to a **continuation value** $\langle \kappa \mid\mid s \rangle$, and then evaluates the body expression e in the extended environment $\sigma[x \mapsto \langle \kappa \mid\mid s \rangle]$.

Function Application

```
def step(k: Cont, s: Stack): (Cont, Stack) = (k, s) match
  case (EvalK(env, expr, k), s) => expr match
    ...
    case App(f, e) => (EvalK(env, f, EvalK(env, e, AppK(k))), s)
  ...
  case (AppK(k), a :: f :: s) => f match
    case CloV(p, b, fenv) => (EvalK(fenv + (p -> a), b, k), s)
    case ContV(k1, s1)    => (k1, a :: s1)
    case v                => error(s"not a function: ${v.str}")
```

$$\langle \kappa \mid\mid s \rangle \rightarrow \langle \kappa \mid\mid s \rangle$$

$$
\begin{array}{lllll}
\texttt{App}_1 & \langle (\sigma \vdash e_1(e_2)) :: \kappa \mid\mid s \rangle & \rightarrow & \langle (\sigma \vdash e_1) :: (\sigma \vdash e_2) :: (@) :: \kappa \mid\mid s \rangle \\
\texttt{App}_{2,\lambda} & \langle (@) :: \kappa \mid\mid v_2 :: \langle \lambda x.e, \sigma \rangle :: s \rangle & \rightarrow & \langle (\sigma[x \mapsto v_2] \vdash e) :: \kappa \mid\mid s \rangle \\
\texttt{App}_{2,\kappa} & \langle (@) :: \kappa \mid\mid v_2 :: \langle \kappa' \mid\mid s' \rangle :: s \rangle & \rightarrow & \langle \kappa' \mid\mid v_2 :: s' \rangle
\end{array}
$$

The new $\texttt{App}_{2,\kappa}$ rule handles when the function expression evaluates to a continuation value $\langle \kappa' \mid\mid s' \rangle$. It changes the control flow to the continuation κ' with the given argument value v_2 and the value stack s'.

Example 1

Let's interpret the expression $2 * (\texttt{vcc x; } (3 + \texttt{x}(5)))$:

$$
\begin{array}{ll}
 & \langle\ (\varnothing \vdash 2 * (\texttt{vcc x; } (3 + \texttt{x}(5)))) :: \square \quad || \quad \blacksquare \quad \rangle \\
(\texttt{Mul}_1) \rightarrow & \langle\ (\varnothing \vdash 2) :: (\varnothing \vdash (\texttt{vcc x; } (3 + \texttt{x}(5)))) :: (*) :: \square \quad || \quad \blacksquare \quad \rangle \\
(\texttt{Num}) \rightarrow & \langle\ (\varnothing \vdash (\texttt{vcc x; } (3 + \texttt{x}(5)))) :: (*) :: \square \quad || \quad 2 :: \blacksquare \quad \rangle \\
(\texttt{Vcc}) \rightarrow & \langle\ (\sigma_0 \vdash (3 + \texttt{x}(5))) :: (*) :: \square \quad || \quad 2 :: \blacksquare \quad \rangle \\
(\texttt{Add}_1) \rightarrow & \langle\ (\sigma_0 \vdash 3) :: (\sigma_0 \vdash \texttt{x}(5)) :: (+) :: (*) :: \square \quad || \quad 2 :: \blacksquare \quad \rangle \\
(\texttt{Num}) \rightarrow & \langle\ (\sigma_0 \vdash \texttt{x}(5)) :: (+) :: (*) :: \square \quad || \quad 3 :: 2 :: \blacksquare \quad \rangle \\
(\texttt{App}_1) \rightarrow & \langle\ (\sigma_0 \vdash \texttt{x}) :: (\sigma_0 \vdash 5) :: (@) :: (+) :: (*) :: \square \quad || \quad 3 :: 2 :: \blacksquare \quad \rangle \\
(\texttt{Id}) \rightarrow & \langle\ (\sigma_0 \vdash 5) :: (@) :: (+) :: (*) :: \square \quad || \quad \langle \kappa_0 \ || \ s_0 \rangle :: 3 :: 2 :: \blacksquare \quad \rangle \\
(\texttt{Num}) \rightarrow & \langle\ (@) :: (+) :: (*) :: \square \quad || \quad 5 :: \langle \kappa_0 \ || \ s_0 \rangle :: 3 :: 2 :: \blacksquare \quad \rangle \\
(\texttt{App}_{2,\kappa}) \rightarrow & \langle\ (*) :: \square \quad || \quad 5 :: 2 :: \blacksquare \quad \rangle \\
(\texttt{Mul}_2) \rightarrow & \langle\ \square \quad || \quad 10 :: \blacksquare \quad \rangle
\end{array}
$$

$$
\text{where} \begin{cases} \sigma_0 = [\texttt{x} \mapsto \langle \kappa_0 \ || \ s_0 \rangle] \\ \kappa_0 = (*) :: \square \\ s_0 = 2 :: \blacksquare \end{cases}
$$

Example 2

Let's interpret the expression $(\lambda \mathtt{x}.(\mathtt{vcc\ r};\ \mathtt{r(x + 1) * 2}))(3)$:

$$
\begin{array}{rll}
 & \langle\ (\varnothing \vdash (\lambda \mathtt{x}.(\mathtt{vcc\ r};\ \mathtt{r(x + 1) * 2}))(3)) :: \square & ||\ \blacksquare\ \rangle \\
\xrightarrow[(\mathtt{App}_{2,\lambda})]{}{}^{*} & \langle\ (\mathtt{@}) :: \square & ||\ 3 :: \langle \lambda \mathtt{x}.e_0, \varnothing \rangle :: \blacksquare\ \rangle \\
\xrightarrow[(\mathtt{Vcc})]{} & \langle\ (\sigma_0 \vdash \mathtt{vcc\ r};\ \mathtt{r(x + 1) * 2}) :: \square & ||\ \blacksquare\ \rangle \\
\xrightarrow[(\mathtt{Mul}_1)]{} & \langle\ (\sigma_1 \vdash \mathtt{r(x + 1) * 2}) :: \square & ||\ \blacksquare\ \rangle \\
\xrightarrow[(\mathtt{App}_1)]{} & \langle\ (\sigma_1 \vdash \mathtt{r(x + 1)}) :: (\sigma_1 \vdash \mathtt{2}) :: (\mathtt{*}) :: \square & ||\ \blacksquare\ \rangle \\
\xrightarrow{} & \langle\ (\sigma_1 \vdash \mathtt{r}) :: (\sigma_1 \vdash \mathtt{x + 1}) :: (\mathtt{@}) :: (\sigma_1 \vdash \mathtt{2}) :: (\mathtt{*}) :: \square & ||\ \blacksquare\ \rangle \\
\xrightarrow[(\mathtt{App}_{2,\kappa})]{}{}^{*} & \langle\ (\mathtt{@}) :: (\sigma_1 \vdash \mathtt{2}) :: (\mathtt{*}) :: \square & ||\ 4 :: \langle \square\ ||\ \blacksquare \rangle :: \blacksquare\ \rangle \\
\xrightarrow{} & \langle\ \square & ||\ 4 :: \blacksquare\ \rangle
\end{array}
$$

where $\begin{cases} e_0 = \mathtt{vcc\ r};\ \mathtt{r(x + 1) * 2} \\ \sigma_0 = [\mathtt{x} \mapsto 3] \\ \sigma_1 = \sigma_0[\mathtt{r} \mapsto \langle \square\ ||\ \blacksquare \rangle] \end{cases}$

Here, `r` acts as a **return** statement: `r(x + 1)` directly returns `4` to the caller, skipping `* 2`.

Contents

1. First-Class Continuations

2. KFAE – FAE with First-Class Continuations
    - Concrete/Abstract Syntax

3. Interpreter and Small-Step Semantics for KFAE
    - Recall: Interpreter and Small-Step Semantics for FAE
    - Interpreter and Small-Step Semantics for KFAE
    - First-Class Continuations
    - Function Application
    - Example 1
    - Example 2

4. Control Statements

Control Statements

Many real-world programming languages support **control statements** to change the **control-flow** of a program.

For example, C++ supports `break`, `continue`, and `return` statements:

```
int sumEvenUntilZero(int xs[], int len) {
  if (len <= 0) return 0;           // directly return 0 if len <= 0
  int sum = 0;
  for (int i = 0; i < len; i++) {
    if (xs[i] == 0) break;          // stop the loop if xs[i] == 0
    if (xs[i] % 2 == 1) continue;   // skip the rest if xs[i] is odd
    sum += xs[i];
  }
  return sum;                       // finally return the sum
}
int xs[] = {4, 1, 3, 2, 0, 6, 5, 8};
sumEvenUntilZero(xs, 8);            // 4 + 2 = 6
```

Let's represent them using **first-class continuations**!

Control Statements

- `return` statement:

```
x => body
```

means

```
x => { vcc return;
  body // return(e) directly returns e to the caller
}
```

- `break` and `continue` statements:

```
while (cond) body
```

means

```
{ vcc break;
  while (cond) { vcc continue;
    body // continue(e)/break(e) jumps to the next/end of the loop
  }
}
```

Control Statements

PLRG

We can represent other control statements similarly, but think for yourself!

- `exception` in Python

```
try:
    x = y / z
except ZeroDivisionError:
    x = 0
```

- `generator` in JavaScript

```
const foo = function* () { yield 'a'; yield 'b'; yield 'c'; };
let str = '';
for (const c of foo()) { str = str + c; }
str // 'abc'
```

- coroutines in `Kotlin`
- `async`/`await` in `C#`
- ...

Summary

PLRG

1. First-Class Continuations

2. KFAE – FAE with First-Class Continuations
   - Concrete/Abstract Syntax

3. Interpreter and Small-Step Semantics for KFAE
   - Recall: Interpreter and Small-Step Semantics for FAE
   - Interpreter and Small-Step Semantics for KFAE
   - First-Class Continuations
   - Function Application
   - Example 1
   - Example 2

4. Control Statements

Exercise #10

https://github.com/ku-plrg-classroom/docs/tree/main/cose212/kfae

- Please see above document on GitHub:
  - Implement `step` function.

- It is just an exercise, and you **don't need to submit** anything.
- However, some exam questions might be related to this exercise, so we **strongly recommend** solving it by yourself.

Next Lecture

- Compiling with Continuations

Jihyeok Park
jihyeok_park@korea.ac.kr
https://plrg.korea.ac.kr